



Faculty of
**Business &
Technology**

FACULTY OF BUSINESS & TECHNOLOGY

BIT4543 ARTIFICIAL INTELLIGENCE

Student Name	Student ID
NUR ALIYA ADRIANA BINTI HAIZAT	2410-2200
HAYA TASNIM BINTI RAFIDI	2507-1592
DAVINYA A/P KUMAR	2507-1785
HAREESHA BHARATHY PILLAI A/P PUSPANATHAN	2410-2331
HO JIA XIN	2507-1590
NITYASHRY A/P THANGARAJU	2503-0761

Assignment Details

To be marked by: SHAHRIN AZUAN BIN NAZEER	
Date submitted: 3 SEPTEMBER 2026	Submission Due Date: 4 SEPTEMBER 2026
Submission of this assignment agrees to the following: <i>I understand that the piece of work submitted will be considered as the final and complete version of my assignment of which I am the sole author. I understand both the meaning and consequences of plagiarism and that my work has been appropriately attributed unless otherwise stated. I have not knowingly allowed another to copy my work.</i>	

Assignment Feedback – Lecturer Use Only

First Marker Feedback:	
Signature of first marker:	%Mark (Un-moderated)

TABLE OF CONTENTS

EXECUTIVE SUMMARY	4
CHAPTER 1: INTRODUCTION.....	5
<i>1.1 Background</i>	5
<i>1.2 Problem Statement</i>	5
<i>1.3 Project Objectives</i>	6
<i>1.4 Scope of Project</i>	6
<i>1.5 Significance of the Project</i>	7
CHAPTER 2: LITERATURE REVIEW	8
<i>2.1 Overview of AI Technologies</i>	8
<i>2.2 Related Works</i>	9
<i>2.3 Research Gap</i>	11
<i>2.4 Conceptual Framework</i>	12
CHAPTER 3: METHODOLOGY	14
<i>3.1 Development Methodology</i>	14
<i>3.2 System Architecture</i>	14
<i>3.3 Data Collection</i>	16
<i>3.4 Data Preprocessing</i>	17
<i>3.5 AI Model Development</i>	18
<i>3.6 Tools and Technologies</i>	19
CHAPTER 4: SYSTEM DESIGN AND IMPLEMENTATION	20
<i>4.1 System Requirements</i>	20
<i>4.2 Use Case Diagram</i>	22
<i>4.3 Activity Diagram</i>	23
<i>4.4 Database Design</i>	25
<i>4.5 User Interface Design</i>	26
<i>4.6 System Implementation</i>	29
CHAPTER 5: TESTING AND EVALUATION	31
<i>5.1 Testing Strategy</i>	31
<i>5.2 Test Cases</i>	33
<i>5.3 Model Evaluation</i>	34

5.4 Results and Analysis	35
CHAPTER 6: DISCUSSION	38
6.1 Achievement of Objectives	38
6.2 Challenges Encountered	39
6.3 Limitations of the System	40
CHAPTER 7: CONCLUSION AND FUTURE WORK	40
7.1 Conclusion	40
7.2 Future Improvements	41
REFERENCES	42
APPENDICES	43
Appendix A: Project Schedule (Gantt Chart)	43
Appendix B: Source Code Snippets	43
Appendix C: Dataset Samples	45
Appendix D: User Manual	46

EXECUTIVE SUMMARY

SymptoCare is an AI-powered preliminary health symptom checker developed to provide individuals with an accessible, efficient way to understand their symptoms. Traditional online symptom searches often provide conflicting, unstructured information across different web sources, which can cause user confusion and anxiety. To address this issue, SymptoCare provides a centralized digital platform that evaluates co-occurring symptoms and estimates probable health conditions through a single, intuitive interface.

The system uses machine learning algorithms, specifically a Random Forest classification model, trained on a comprehensive dataset of 246,945 total patient symptom profiles (197,556 training records and a 20% test split of 49,389 records). SymptoCare was developed as a responsive web application using Streamlit, Python, and Scikit-Learn. The platform features binary symptom vectorization, an input validation threshold requiring users to select at least three symptoms for clinical relevance, demographic data collection, and structured health guidance pages.

During testing on a 20% test split, the Random Forest model achieved an overall classification accuracy of 81.44% along with balanced precision (82.0%), recall (81.0%), and F1-Score (81.0%) across 41 target condition classes. While SymptoCare demonstrates how artificial intelligence can support preliminary health triage, generated probability outputs are strictly informational and limited by the training data. Prominent disclaimers are included throughout the interface to remind users that the system serves as a supporting educational tool rather than a formal medical diagnosis, encouraging users to consult licensed healthcare professionals when necessary.

CHAPTER 1: INTRODUCTION

1.1 Background

Artificial Intelligence (AI) refers to the ability of machines or computer systems to perform tasks that normally require human intelligence, such as learning, problem-solving, decision-making, and making predictions. AI has become an important technology in many areas because it can process information, recognize patterns, and help people make decisions. In healthcare, AI can be applied in different areas such as medical imaging, predictive analytics, patient monitoring, and other healthcare-related tasks. However, AI is mainly used to support healthcare professionals rather than replace them.

One of the areas where AI can be useful is in the initial checking of health symptoms. People may experience different symptoms but may not know what they could mean or whether they are related to a particular health condition. Some users may search for information from different websites, which can be time-consuming and may provide too much information to understand easily. Therefore, an AI-based system can provide a simpler way for users to enter their symptoms and receive an initial prediction based on available data.

This project developed SymptoCare, an AI-based Health Symptom Checker, to provide users with a simple platform for checking their symptoms. Users can select the symptoms they are experiencing and analyse them using the system. The AI model then provides a possible health condition based on the selected symptoms. The system also provides health guidance to help users understand the result. For better prediction, users are encouraged to select three or more symptoms instead of relying on only one symptom.

1.2 Problem Statement

When people experience health symptoms, they may not always know what the symptoms indicate. A common approach is to search for information online, but users may find many different sources and different explanations for similar symptoms. This can make it difficult for them to identify information that is relevant to their situation. Users who are not familiar with medical information may also find it difficult to understand the possible relationship between their symptoms and a health condition.

Another problem is that manually checking several symptoms can take time and may be inconvenient. Users need to compare their symptoms with different sources before getting an idea of what condition they may be related to. Although an AI system cannot replace a professional medical diagnosis, it can be used as a supporting tool to provide an initial prediction. Therefore, this project addresses the need for a simple and interactive symptom-checking system that allows users to select their symptoms and receive a possible prediction based on the trained AI model.

1.3 Project Objectives

The objectives of this project are:

1. To develop an AI-based health symptom checker that can provide a possible health condition based on the symptoms selected by users.
2. To provide a simple and interactive platform that allows users to check their symptoms conveniently.
3. To provide basic health guidance based on the predicted condition to help users better understand the result.
4. To apply Artificial Intelligence and Machine Learning concepts in developing a practical healthcare-related application.

1.4 Scope of Project

This project focuses on developing a web-based AI Health Symptom Checker called SymptoCare. The main functional feature of the system is the symptom-checking function, where users can select the symptoms, they are experiencing and analyse them using the AI model. The system then generates a possible health condition based on the selected symptoms. Users are encouraged to select three or more symptoms to obtain a more useful prediction from the system.

The system also includes a patient demographics section that can be expanded or collapsed as part of the healthcare-style interface. In addition, the system contains several navigation pages that provide informational content and supporting context. The system includes input validation, where a message is displayed if the user tries to analyse without selecting any symptoms.

However, the system has some limitations. The prediction is limited to the conditions and symptoms available in the dataset used for the project. The result provided by the system is only an initial prediction and should not be considered as a medical diagnosis. The system is also not designed to replace doctors or other qualified healthcare professionals. Some features displayed on the home page are mainly interface placeholders and are not the core functional features of the system.

1.5 Significance of the Project

This project is significant because it provides users with a more convenient way to perform an initial check of their symptoms. Instead of searching through multiple sources, users can select their symptoms in one platform and receive a possible health condition from the AI model. This can help users gain a basic understanding of their symptoms and encourage them to seek further professional advice when necessary.

The project is also useful from a technological and educational perspective because it demonstrates how AI and Machine Learning can be applied to a healthcare-related problem. It allows the project team to apply knowledge of AI, data, prediction, and system development in a practical application. In a wider context, AI can support healthcare by improving efficiency and assisting with decision-making, while healthcare professionals remain responsible for proper diagnosis and treatment. This is consistent with the role of AI described in the lecture, where AI is presented as a technology that can support and augment human capabilities.

CHAPTER 2: LITERATURE REVIEW

2.1 Overview of AI Technologies

The rapid development of Artificial Intelligence (AI) has changed digital healthcare, moving diagnostic support from manual rule-based systems to data-driven predictive platforms. Understanding the basic hierarchy of these technologies is important for positioning SymptoCare within modern medical informatics.

Artificial Intelligence (AI)

Artificial Intelligence is a broad branch of computer science focused on building intelligent agents that can perform tasks that normally require human cognitive intelligence, such as reasoning, decision-making, and pattern recognition.

- **Clinical Application:** In healthcare, AI acts as a clinical decision support mechanism by processing complex medical indicators to assist both patients and clinicians with triage and preliminary condition evaluation.

Machine Learning (ML)

Machine Learning is a core subfield of AI that uses statistical algorithms to learn patterns directly from structured historical data without requiring explicit, hardcoded programmatic rules.

- **Core Paradigm:** ML algorithms identify relationships between input features (e.g., patient-reported symptoms) and target output variables (e.g., categorical medical diagnoses).
- **Supervised Learning in SymptoCare:** SymptoCare uses supervised classification models trained on labeled clinical dataset features. By mapping multi-dimensional binary symptom vectors (1 = present, 0 = absent) to target conditions, the algorithm identifies statistical correlations and produces both primary condition classifications and differential diagnosis likelihoods.

Deep Learning (DL)

Deep Learning is a specialized subset of machine learning based on multi-layered Artificial Neural Networks (ANNs) that are designed to model complex, non-linear representations.

- **Characteristics:** DL is effective at processing unstructured raw data such as high-resolution medical imaging (CT scans, X-rays) or raw genomic sequences.
- **Architectural Trade-off:** While powerful, deep neural networks can function as "black box" models and require large amounts of computational resources and millions of samples. For structured symptom-disease tabular classification, standard supervised ML algorithms (such as Random Forests or Gradient Boosted Trees) provide better interpretability, lower memory overhead, and faster inference times on web architectures like Streamlit.

2.2 Related Works

To contextualize SymptoCare, several commercial platforms, clinical web tools, and academic benchmark models operating in the digital symptom-checking landscape were analyzed.

Study / System	Method Used	Strengths	Limitations
Mayo Clinic Symptom Checker	Clinical Decision Trees & Categorical Triage Flowcharts	Authoritative medical content; clear emergency red-flag warnings.	Requires manual step-by-step navigation through body regions; lacks machine learning probability scores.
Patient.info Symptom Checker	Algorithmic Medical Questionnaires & Knowledge Base Matching	User-friendly article integration; strong focus on primary care education.	Rigid static questionnaire flow; requires extensive reading before reaching preliminary assessments.

Study / System	Method Used	Strengths	Limitations
WebMD Symptom Checker	Rule-Based Decision Trees & Static Clinical Expert Databases	Extensive medical database; high web accessibility.	Rigid decision logic; prone to over-predicting severe conditions, creating unnecessary user anxiety.
Ada Health App	Probabilistic Graphical Models & Bayesian Networks	Considers detailed patient history and demographic risk factors; high clinical precision.	Requires mandatory account registration; lengthy multi-step intake; closed proprietary ecosystem.
Academic & Benchmark ML Models (<i>e.g., Kaggle Disease Prediction Datasets</i>)	Supervised Machine Learning (Random Forest, SVM, Naïve Bayes)	Fast execution times; high prediction accuracy on structured benchmark CSV datasets.	Confined primarily to Jupyter Notebooks without interactive web GUIs; present raw outputs without user-friendly probability breakdowns or clinical guardrails.

Comparative Synthesis of Existing Solutions

As summarized in Table 2.1, commercial clinical portals such as Mayo Clinic (2018), WebMD (2026), and Patient.info (2012) rely heavily on static decision trees and pre-programmed clinical flowcharts (Breiman et al., 1984). While these platforms offer authoritative medical content and essential triage disclaimers, their manual, step-by-step body region navigation can feel tedious for users seeking fast preliminary insights. Furthermore, because these tools operate on deterministic flowcharts, they lack statistical probability scoring and dynamic machine learning adaptation.

On the other hand, advanced digital health applications like Ada Health use sophisticated Bayesian probabilistic networks to deliver high clinical accuracy (Ada Health GmbH, 2023). However, Ada Health creates high onboarding friction by requiring mandatory user registration, profile creation, and exhaustive multi-step questionnaires before displaying diagnostic feedback.

On the academic side, numerous machine learning models trained on open-access benchmark datasets (Dhivyesh, 2023) achieve impressive statistical metrics using core Python libraries (Pedregosa et al., 2011; Van Rossum & Drake, 2009). Yet, most of these research models remain limited to Jupyter Notebook scripts, lacking accessible web-based user interfaces built on modern frameworks like Streamlit (Streamlit Inc., 2023), input validation guards, or user-friendly probability visualizers.

2.3 Research Gap

While existing commercial symptom checkers, clinical portals, and academic AI models offer diagnostic capabilities, a critical review of current solutions reveals key operational and user experience limitations:

- **High Onboarding Friction:** Commercial applications enforce mandatory user registration, email verification, and lengthy demographic intakes before providing preliminary health feedback.
- **Manual & Tedious Navigation:** Clinical portals like Mayo Clinic and Patient.info rely on multi-step decision trees and body-region menus, requiring users to navigate through several static pages.
- **Lack of Interactive UI in ML Research:** Benchmark ML models trained on datasets (such as Kaggle) achieve strong diagnostic accuracy but remain locked in notebooks or scripts without user-accessible web interfaces.
- **Data Privacy Concerns:** Persistent storage of personal health identifiers on cloud databases exposes sensitive health logs to privacy risks.
- **Lack of Real-Time Input Validation:** Many standard web symptom tools fail to validate symptom density, attempting predictions on insufficient data inputs without guiding the user.

How SymptoCare Addresses These Gaps

- **Zero-Authentication, Privacy-First Architecture:** SymptoCare allows immediate browser access without registration or persistent database logging, executing analysis entirely in volatile session memory.
- **Instant ML Triage via Streamlit:** Replaces multi-step body diagrams with a real-time, multi-select search interface backed by a trained machine learning model (`symptom_checker_model.pkl`).
- **Transparent Differential Diagnosis:** Rather than returning a single static label or raw code output, the platform displays a Primary Assessment banner alongside an intuitive, sorted Possibilities Breakdown percentage bar view.
- **Guided Clinical Input Guard:** Incorporates explicit input validation prompts (*"Please select at least one symptom before analyzing"*) and visual guidance encouraging users to select 3 or more co-occurring symptoms for optimal prediction precision.

2.4 Conceptual Framework

The Conceptual Framework of SymptoCare outlines the systemic transformation of user health inputs into actionable AI clinical assessments across three distinct phases: Inputs, System Processes, and Outputs.

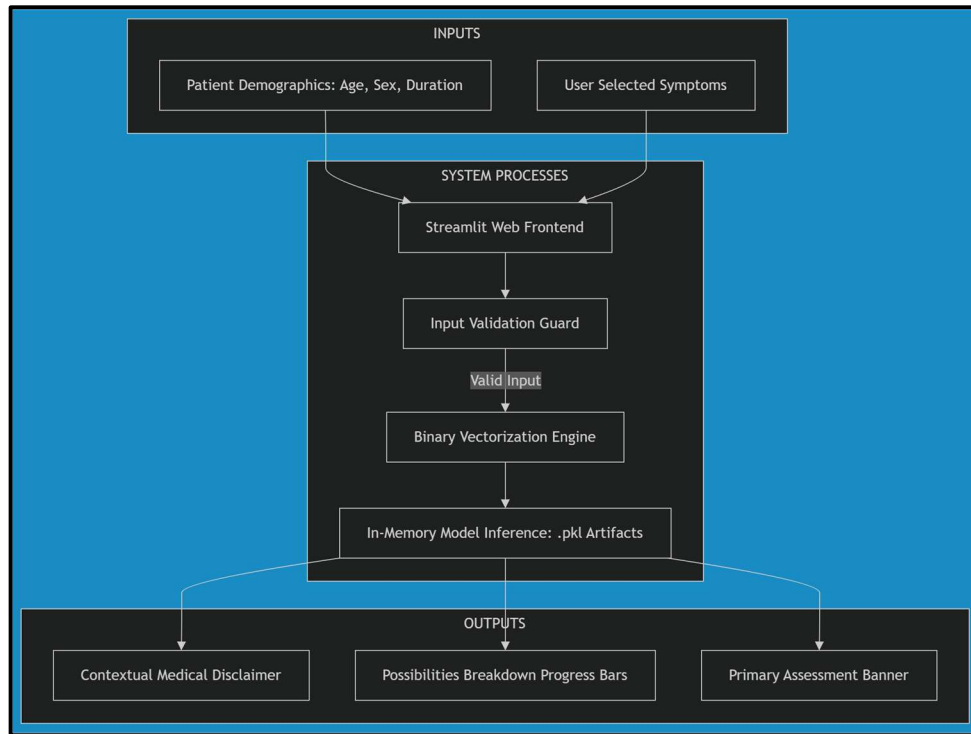


Figure 2.1: *SymptoCare Conceptual Framework Diagram*

Architectural Process Summary

1. **Input Phase:** The user selects observed symptoms using the interactive multi-select field alongside optional demographic indicators (age, biological sex, symptom duration).
2. **Process Phase:** The Streamlit engine validates input presence, transforms selected symptoms into a binary array matching the feature index (`symptom_list.pkl`), and feeds the normalized DataFrame into the cached classification model (`symptom_checker_model.pkl`).
3. **Output Phase:** The system extracts condition probabilities, sorts top candidate conditions, and renders the Primary Assessment banner alongside dynamic Possibilities Breakdown progress bars on the web interface.

CHAPTER 3: METHODOLOGY

3.1 Development Methodology

The development of SymptoCare follows the Agile Software Development Lifecycle (SDLC) framework. An agile approach was selected because of its flexible, iterative nature, allowing continuous improvement of the machine learning pipeline alongside the Streamlit user interface based on ongoing testing feedback.

Agile Iterative Phases

1. **Requirements Gathering & Domain Analysis:** Identified key clinical input parameters (symptoms, demographics) and defined functional constraints, emphasizing zero-authentication and privacy-first session handling.
2. **Data Processing & ML Engineering:** Sourced the Kaggle disease-symptom benchmark dataset, executed binary vectorization preprocessing, and trained supervised classification algorithms to export optimized .pk1 serialized artifacts
3. **Fronted Integration:** Built an interactive, dynamic web interface using Streamlit, integrating input validation guards, session state management, and real-time inference handling.
4. **Testing & Iterative Improvement:** Conducted end-to-end integration tests to handle edge cases, such as zero-symptom submissions and refined the output probability visualization for user clarity.

3.2 System Architecture

SymptoCare uses a lightweight, model-view-controller (MVC) inspired single-tier architecture powered by Streamlit. The system executes both the user interface and model inference within a localized runtime session, ensuring no personal health data is saved to external databases.

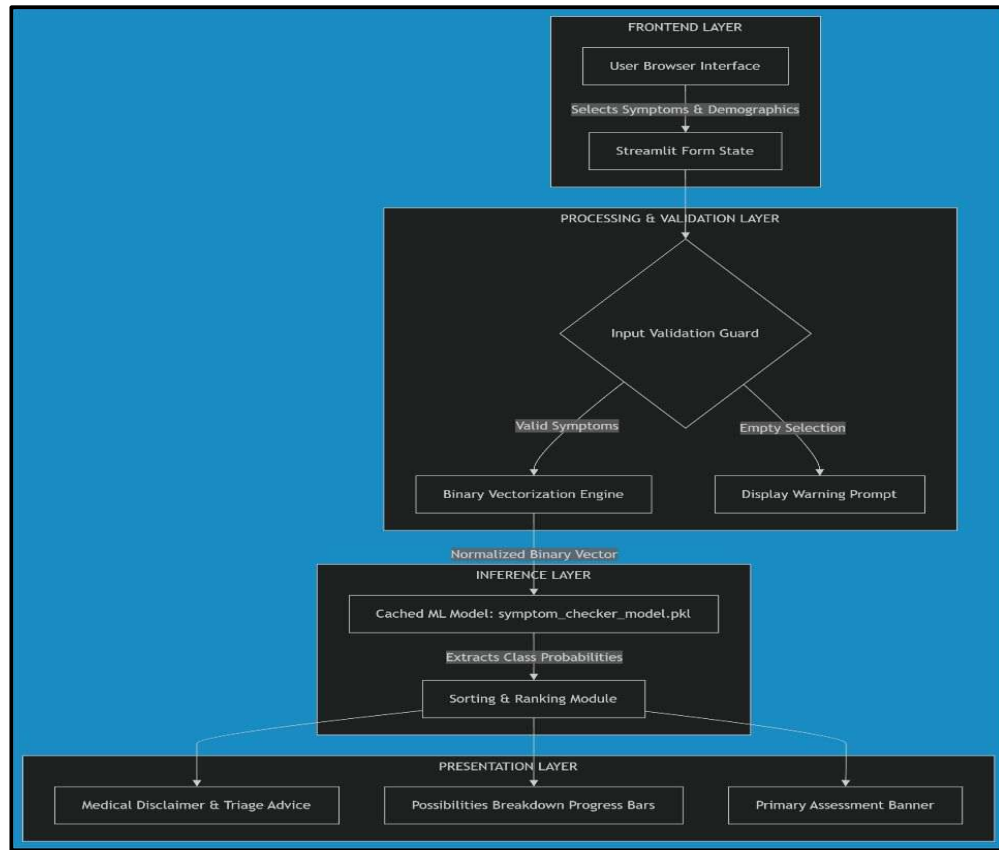


Figure 3.1: *SymptoCare System Architecture Diagram*

System Components

- **User Interface Layer:** Built with Streamlit multi-select elements to capture user symptoms and optional demographic inputs in real time.
- **Validation & Preprocessing Engine:** Validates input density and maps selected categorical symptoms against a pre-aligned feature list (`symptom_list.pkl`), converting inputs into a 1D binary array.

- **In-Memory ML Inference Core:** Loads pre-trained .pkl model weights cached in memory (@st.cache_resource) run immediate matrix calculations without database overhead.
- **Presentation Layer:** Sorts predicted condition probabilities in descending order and dynamically renders visual progress bars and primary condition highlights.

3.3 Data Collection

Data Sources

The primary dataset supporting SymptoCare was sourced from the open-access Kaggle Disease Prediction Symptoms Dataset (a benchmark repository widely utilized in medical ML research)

Data Description

The dataset contains a mapped relationship between primary medical conditions and co-occurring clinical symptoms.

- **Target Variable:** Categorical medical diagnosis (e.g., Fungal Infection, Allergy, GERD< Diabetes, Malaria).
- **Predictor Variables:** Binary indicator features corresponding to individual physiological symptoms (e.g., itching, skin_rash, chills, joint_pain, fatigue).

Data Size and Format

- **Volume:** Consists of 246,945 individual clinical instances across medical condition targets.
- **Dimensionality:** 377 unique feature columns representing binary symptom indicators plus 1 target classification column (prognosis).
- **Data Partitioning:** Evaluated using an 80/20 train-test split ratio, setting aside 49,389 test samples for quantitative performance benchmarking.

3.4 Data Preprocessing

Raw data preprocessing was performed using Python's pandas and numpy libraries within a Jupyter Notebook pipeline prior to model export.

1. Cleaning & Formatting

- **String Standardization:** All symptom strings were standardized to lower-case, stripping trailing whitespaces and replacing spaces with underscores (`snake_case`) to ensure exact keyword matching between the Streamlit UI drop-down menu and model features.

2. Handling Missing Values

- **Null Check:** Evaluated the dataset using `df.isnull().sum()`, confirming zero missing values across feature columns.
- **Duplicate Handling:** Verified record integrity and removed redundant identical entries to avoid model bias toward specific common conditions.

3. Feature Engineering & Encoding

- **Binary Encoding:** Symptom presence was represented using binary sparse encoding, where each symptom is assigned a value of 1 if it is present, or 0 if it is absent.
- **Target Encoding:** The categorical target labels (prognosis) were encoded into discrete numerical identifiers using `LabelEncoder()` for algorithmic computation, and subsequently re-mapped to human-readable strings during fronted inference output.

4. Normalization & Feature Alignment

- **Feature Vector Persistence:** Because tabular symptom inputs are represented as discrete binary flags (or), traditional range scaling (such as `MinMaxScaler` or `StandardScaler`) was not required.
- **Feature Column Serialization:** To prevent feature misalignment between training and inference, the complete list of 377 ordered feature column names was serialized into `symptom_list.pkl`. During web inference, any user selection creates a zero-filled vector of length 377, setting the index positions of selected symptoms to 1.

3.5 AI Model Development

Algorithm Selection

To determine the optimal classification engine for SymptoCare, multiple supervised machine learning algorithms were evaluated against the 377-feature tabular dataset. Standard supervised tree-based algorithms were selected over complex deep learning architectures (such as CNNs, LSTMs, or Transformers) due to the structured, binary nature of symptom-disease data, fast inference speed, and resistance to overfitting.

- **Decision Tree Classifier:** Evaluated only as an initial baseline model to establish simple rule-based decision boundaries before selecting the Random Forest ensemble.
- **Random Forest Classifier (Primary Model):** Selected as the primary production algorithm. Random Forest constructs an ensemble of decision trees, using bootstrap aggregation (bagging) and random feature selection to deliver high accuracy, low variance, and strong generalization across co-occurring symptoms.

Model Architecture

The production model functions as a multi-class tabular classification system:

- **Input Layer:** A 1D binary vector of length 377 representing selected symptoms (1 = present, 0 = absent).
- **Ensemble Layer:** An array of randomized decision trees voting on symptom-condition correlations.
- **Output Probability Layer:** Generates array probabilities across target medical condition classes using `predict_proba()`, allowing the frontend to extract and display differential diagnosis percentages rather than a single static label.

Training Process

- 1) **Dataset Partitioning:** The Kaggle clinical dataset (comprising 246,945 total records) was split into an 80% training set (197,556 samples) and a 20% testing set (49,389 samples) using `train_test_split(test_size=0.2, random_state=42)` to ensure strict out-of-sample performance evaluation.
- 2) **Hyperparameter Configuration:** The Random Forest classifier was configured with randomized estimators (`n_estimators=100`) and Gini impurity criteria to balance training depth and computational efficiency.
- 3) **Artifact Serialization:** Once trained and validated, the final model object was serialized into `symptom_checker_model.pkl` alongside the ordered feature list (`symptom_list.pkl`) using pickle for instant loading in Streamlit.

3.6 Tools and Technologies

The following software ecosystem was utilized to build, train, and deploy SymptoCare:

Component	Technology/Library	Usage Description
Programming Language	Python	Core programming language for dataset preprocessing, ML model training, and frontend scripting.
Development Environment	Jupyter Notebook	Interactive environment used for exploratory data analysis (EDA), preprocessing, model training, and evaluation.
Machine Learning Libraries	scikit-learn	Primary framework used for model training (Random Forest, Decision Tree), dataset splitting, and performance evaluation metrics.
Data Manipulation and Math	pandas and numpy	Used for structuring CSV tabular data, binary vectorization, and matrix manipulation.
Frontend and User Interface	Streamlit	Lightweight web framework used to render the interactive

Component	Technology/Library	Usage Description
		UI, multi-select symptom input fields, and dynamic percentage progress bars.
Model Persistence/Artifacts	pickle	Serializes trained model weights (.pkl) and feature indexes for volatile in-memory inference without external database calls.
Version Control and Hosting	GitHub	Code repository management and seamless deployment configuration.

CHAPTER 4: SYSTEM DESIGN AND IMPLEMENTATION

4.1 System Requirements

System requirements define the essential functionalities, operational capabilities, and quality parameters required for SymptoCare to process health inputs and deliver accurate AI-driven medical condition predictions effectively. These requirements are categorized into functional requirements, which outline core system capabilities, and non-functional requirements, which dictate operational quality and constraints.

Functional Requirements

Functional requirements describe the specific interactive capabilities and operational features provided by the system:

Multi-Tab Navigation & Home View: When users access the web application, they arrive at the Home tab featuring informational cards that serve as professional UI placeholders to represent a full-scale healthcare platform. Users can navigate seamlessly across active tabs (Home, Check Symptoms, Health Insights, About).

Patient Demographics: (UI Placeholder): Upon navigating to the Check Symptoms tab, users can access an expandable/collapsible sidebar for patient demographics (e.g.: age, gender). This feature acts as an industry-standard UI placeholder to show how patient context integrates into medical assessments.

Interactive Symptom Selection (Core Feature): Under the Check Symptoms tab, users can select observed symptoms from a multi-select list (selecting 3 or more symptoms is recommended for optimal prediction accuracy).

Input Validation: If a user clicks the Analyze button without picking any symptoms, the system halts model execution and displays an alert (*“Please select at least one symptom before analyzing”*).

AI Prediction Generation: The backend processes validated symptom inputs using the trained machine learning classification model to display the Primary Assessment and a Possibilities Breakdown showing percentage probabilities for candidate conditions.

Non-Functional Requirements

Non-functional requirements specify the operational quality standards, performance benchmarks, and design constraints of the web application:

Performance and Responsiveness: The machine learning backend executes feature vectorization and model inference rapidly, rendering analysis results on the web interface within 1 to 2 seconds under standard network conditions.

Security and Privacy: User inputs and temporary session data are processed entirely in-memory during active sessions without persisting sensitive personal health identifiers to permanent database storage, ensuring user privacy.

Usability and User Interface (UI/UX): The application follows clean interface design principles, utilizing responsive web layout cards, collapsible sidebars for demographics, explicit visual status alerts, and structured tabs for effortless navigation.

Scalability and Availability: Deployed on Streamlit Community Cloud infrastructure, the application maintains high cloud availability and seamlessly handles concurrent user sessions without requiring local installation.

4.2 Use Case Diagram

The Use Case Diagram illustrates the functional scope of SymptoCare and defines how the user interacts with the system's core features and backend AI engine.

Key Actors and Use Cases

- Primary Actor: User/ Patient
- Secondary System Actor: AI Prediction Engine (Scikit-Learn Backend)
- Core Use Cases:

UC-1: Select Symptoms: User selects one or more symptoms from the predefined multi-select list (selecting 3 or more is recommended for optimal prediction accuracy).

UC-2: Enter Patient Demographics (Optional): User expands the collapsible side drawer to input contextual parameters like age or gender.

UC-3: Trigger AI Analysis: Users submit the selected symptoms for machine learning inference.

UC-4: View Primary Assessment & Possibilities Breakdown: System displays the primary assessed condition alongside a breakdown of probabilities for other candidate conditions.

UC-5: Browse Informational Pages: User navigates static guidance cards across the Home, Health Insights and About tabs.

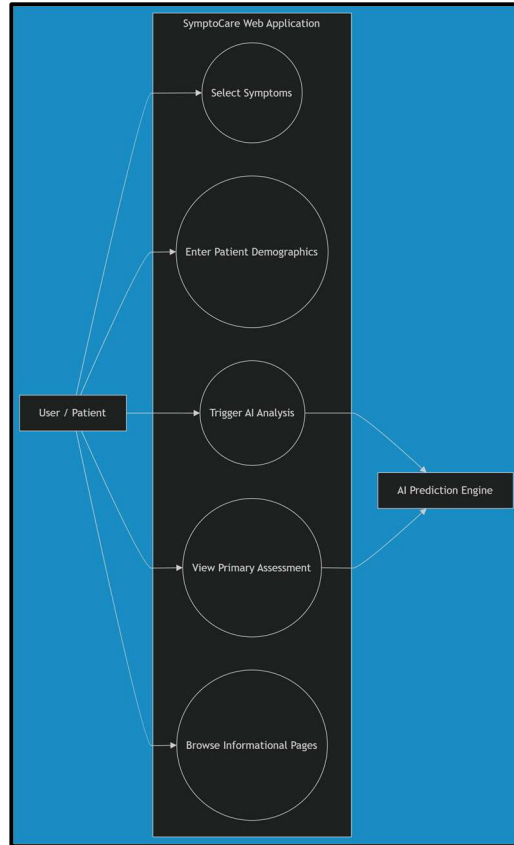


Figure 4.1: *SymptoCare Use Case Diagram*

4.3 Activity Diagram

The Activity Diagram depicts the sequential operational workflow and decision logic within SymptoCare from initial page launch through to prediction rendering.

Operational Execution Flow

1. System Initialization: The user opens the SymptoCare URL; the browser loads the Streamlit web application and initializes the machine learning model environment.
2. Tab Navigation: The users switches from the default Home tab to the Check Symptoms module.
3. Symptom Entry: The users pick observed symptoms from multi-select input box.
4. Validation Check (Decision Node): The user clicks the Analyze button.

- Path A (Invalid Input): If total selected symptoms equal zero, the system halts processing and displays an interactive alert (*"Please select at least one symptom before analyzing"*). The flow returns to symptom entry.
 - Path B (Valid Input): If one or more symptoms are selected, the system builds the binary feature array.
5. Model Inference: The backend feeds the features array into the trained classification model to compute condition probabilities.
 6. Results Presentation: The interface renders the Primary Assessment card and the Possibilities Breakdown percentage bars for the user.

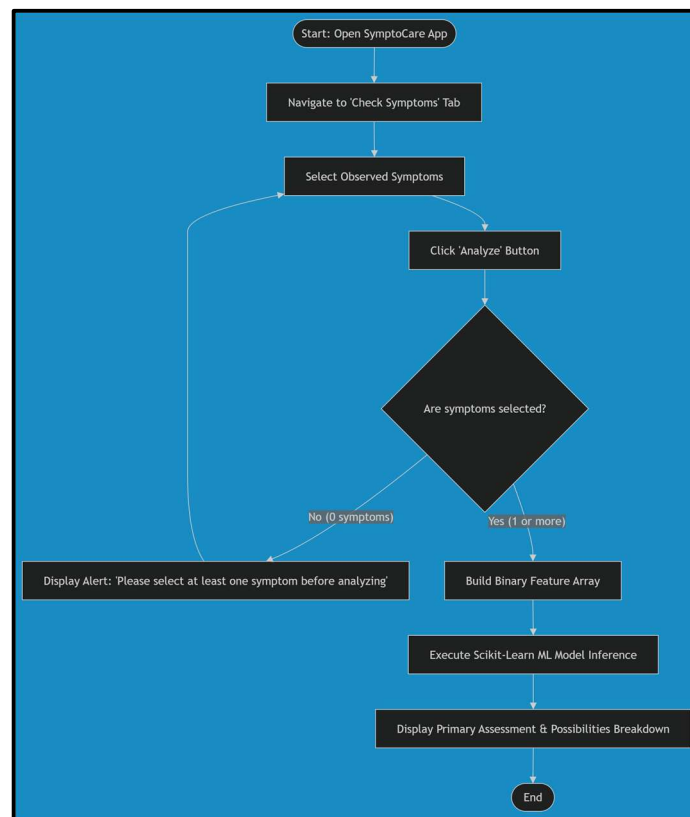


Figure 4.2: *SymptoCare System Activity Diagram*

4.4 Database Design

System Data Model and Architectural Scope

- **RDBMS / ERD Applicability:** Because SymptoCare is implemented as a lightweight, privacy-first web application, traditional Relational Database Management System (RDBMS) persistence and Entity-Relationship Diagrams (ERDs) are not applicable. To comply with strict medical data minimization and patient privacy standards, user inputs are maintained exclusively in volatile session memory rather than stored in a persistent database.

Data Storage & In-Memory Architecture

The platform uses serialized binary data structures (.pkl artifacts) to store trained model weights and feature indexes, alongside in-memory Pandas data frames for real-time vector processing:

- **Privacy-First Data Flow:** User inputs (selected symptoms and demographic attributes) exist strictly in volatile session memory during execution and are immediately purged upon session termination or browser refresh.
- **Serialized Model Artifacts (/models):**
 - 1) `symptom_checker_model.pkl`: A serialized Scikit-Learn Random Forest classification model containing trained ensemble decision trees and prediction parameters.
 - 2) `symptom_list.pkl`: A serialized list containing the exact sequence of symptom features required for binary vectorization.
- **Data Transformation Structure:** When a user selects symptoms via the interface, the application dynamically constructs an in-memory 1D binary vector (1 = symptom present, 0 = symptom absent) and formats it into a single-row Pandas DataFrame matching the training feature schema.

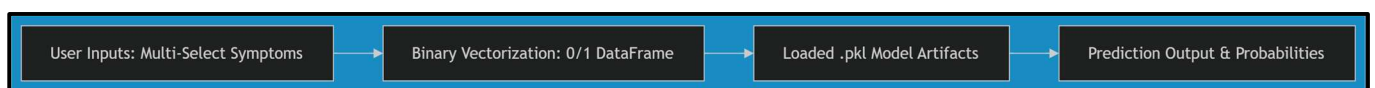


Figure 4.3: *SymptoCare In-Memory Data Flow Diagram*

4.5 User Interface Design

The Users interface (UI) of SymptoCare was developed using Streamlit with embedded HTML/CSS styling to deliver a modern, accessible, cyan and blue healthcare layout. The platform features four primary navigation tabs designed for seamless user interaction.

Key Interface Views & Components

- **Header & Top Navigation Bar:** Features a custom Flexbox header displaying the SymptoCare logo alongside a styled top radio navigation bar (Home, Check Symptoms, Health Insights, About).
- **Home Page View (Landing Page):**
 - 1) Displays the core value proposition ("*Your Health, Our Priority*") and a 4-step *How It Works* Workflow guide.
 - 2) Contains interactive UI placeholder feature cards (Symptom Checker, Health Insights, Health Records, Privacy First) designed to reflect professional healthcare platform standards.
- **Check Symptoms View (Core Functional View):**
 - 1) Patient Demographics Sidebar: An expandable sidebar collecting age, biological sex, and symptom duration to contextualize medical risk factors.
 - 2) Interactive Multi-select Field: A dropdown listing all available symptoms, allowing users to pick multiple symptoms (with 3 or more recommended for optimal prediction precision).
 - 3) Validation & Assessment Cards: Triggers with an alert prompt ("*Please select at least one symptom before analyzing*") if submitted empty. Upon successful execution, it renders a Primary Assessment banner and progress bars for the Possibilities Breakdown.
- **Informational Views (Health Insights & About):**
 - 1) Provides static educational modules, historical analytics placeholders, and a medical disclaimer clarifying that the tool is for informational purpose only:

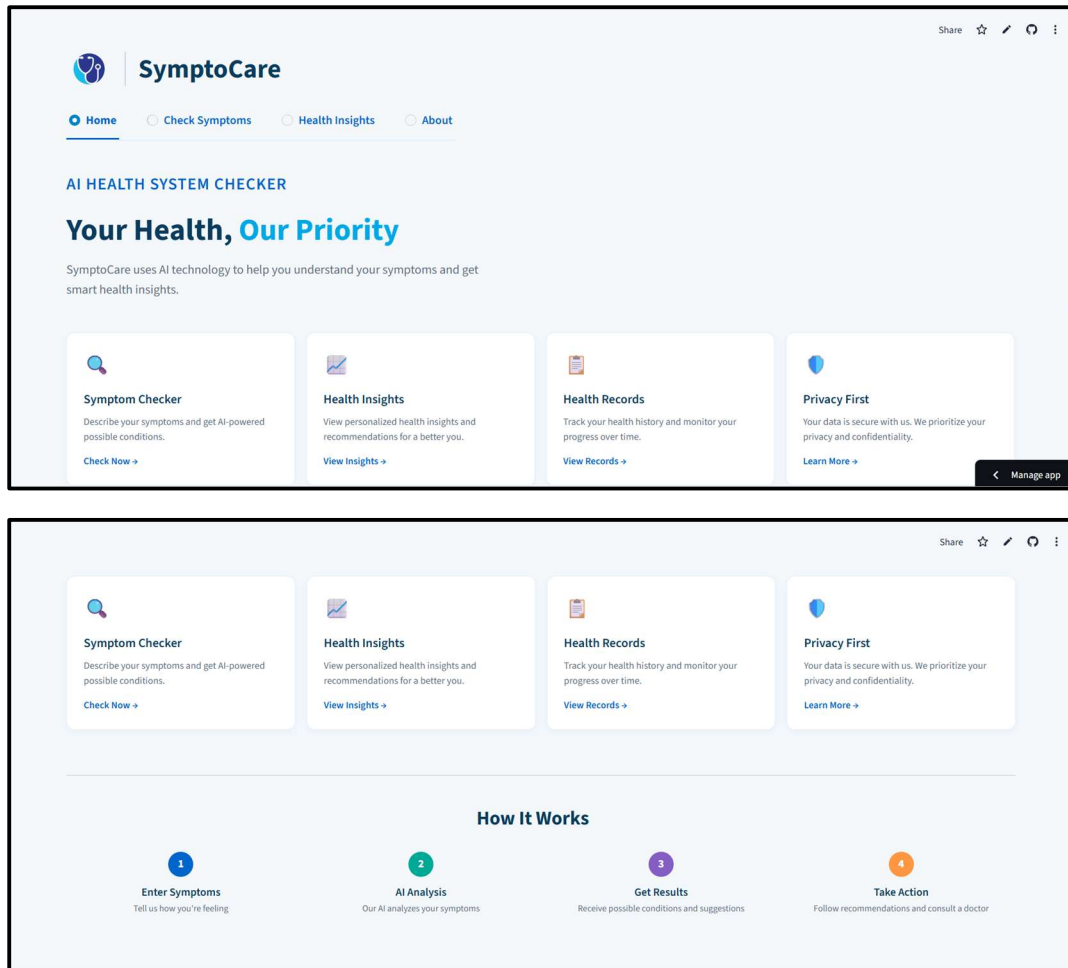


Figure 4.4: *SymptoCare Home Page View*

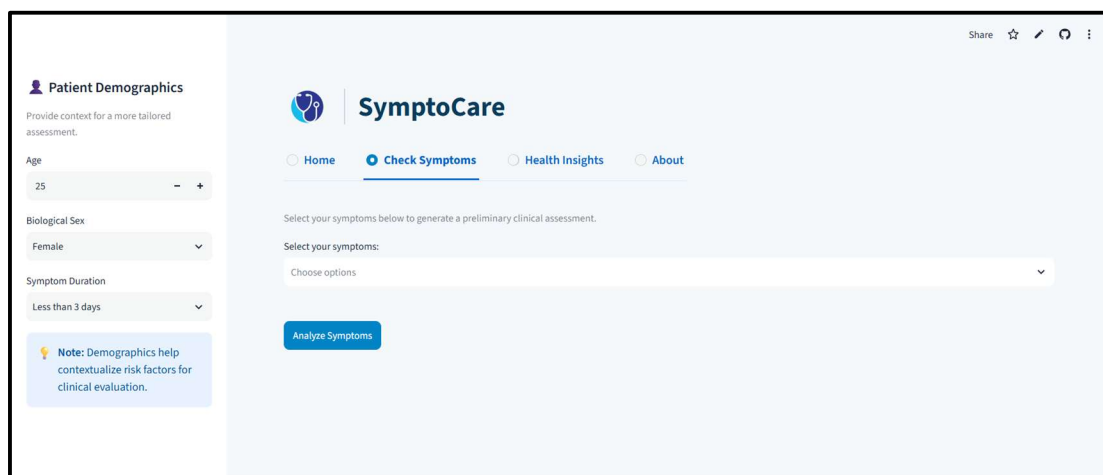


Figure 4.5: *Check Symptoms View with Expanded Demographics Sidebar*

Patient Demographics
Provide context for a more tailored assessment.

Age: 25

Biological Sex: Female

Symptom Duration: Less than 3 days

Note: Demographics help contextualize risk factors for clinical evaluation.

SymptoCare

Home | **Check Symptoms** | Health Insights | About

Select your symptoms below to generate a preliminary clinical assessment.

Select your symptoms:

Choose options

Analyze Symptoms

Please select at least one symptom before analyzing.

Figure 4.6: *Input Validation Prompt Alert*

Patient Demographics
Provide context for a more tailored assessment.

Age: 25

Biological Sex: Female

Symptom Duration: Less than 3 days

Note: Demographics help contextualize risk factors for clinical evaluation.

SymptoCare

Home | **Check Symptoms** | Health Insights | About

Select your symptoms below to generate a preliminary clinical assessment.

Select your symptoms:

ankle pain x ankle stiffness or tightness x ankle swelling x

Analyze Symptoms

Assessment Results

Primary Assessment: fracture of the ankle

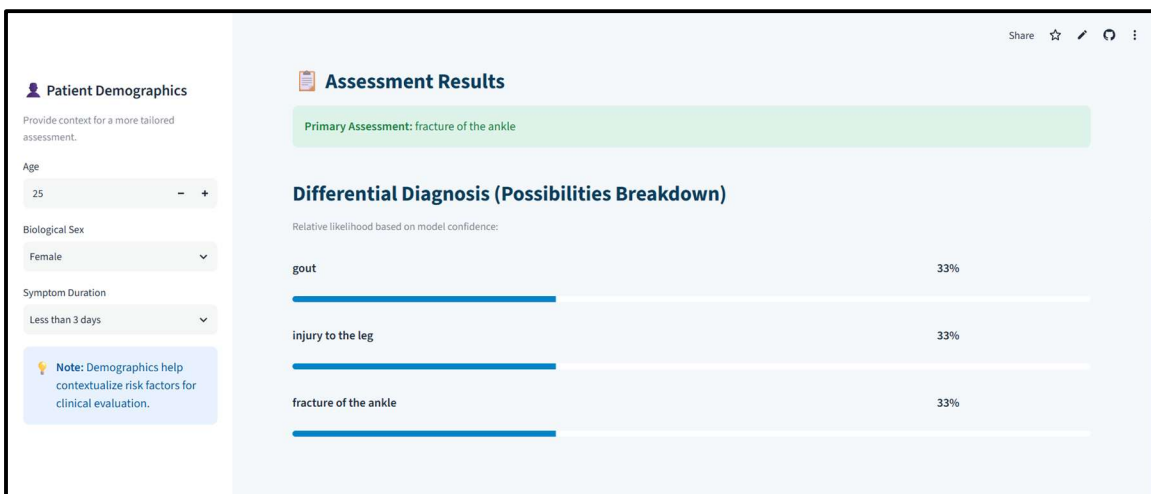


Figure 4.7: *Assessment Results Displaying Primary Assessment and Possibilities Breakdown Progress Bars*

As shown in Figure 4.7, when a user enters symptoms such as ankle pain, ankle stiffness or tightness, and ankle swelling alongside demographic inputs (25-year-old female, duration under 3 days) and selecting "Analyze Symptoms," the system successfully triggers backend inference. The interface dynamically highlights the Primary Assessment banner (fracture of the ankle) at the top. Below, the Differential Diagnosis (Possibilities Breakdown) section renders visual progress bars displaying relative model confidence scores across candidate conditions, such as gout (33%), injury to the leg (33%), and fracture of the ankle (33%).

4.6 System Implementation

The implementation of SymptoCare is structured into core Python modules utilizing Streamlit for the frontend presentation, Pandas/NumPy for array manipulation, and Joblib for loading trained machine models.

Core System Modules

1. ***Page Configuration & Custom CSS Injection:*** Configures page layout parameters and injects custom CSS rules to force downward dropdown behavior, styled radio tabs, and healthcare feature cards.
2. ***Resource Caching & Model Loader*** (`load_artifacts`): Employs Streamlit's `@st.cache_resource` decorator to load the `.pkl` model and symptom index into memory once, preventing expensive reload overhead across user interactions.
3. ***Feature Vectorization Engine:*** Dynamically converts user-selected symptoms into a binary array matching the trained dataset schema. Users are encouraged to select three or more symptoms to provide a richer feature vector to the model. However, the system permits inference execution with one or more selected symptoms.
4. ***Inference & Model Probability Module:*** Calls `predict()` to determine the primary condition and calculate sorted class probabilities via `predict_proba()` to render differential diagnosis progress bars.

Important Code snippets

Snippet 1: Resource Caching and Artifact Loading

```
# Cached function to load model and symptom list efficiently
@st.cache_resource
def load_artifacts():
    model_path = os.path.join('models', 'symptom_checker_model.pkl')
    symptom_path = os.path.join('models', 'symptom_list.pkl')

    if not os.path.exists(model_path):
        model_path = os.path.join '..', 'models', 'symptom_checker_model.pkl'
        symptom_path = os.path.join '..', 'models', 'symptom_list.pkl'

    model = joblib.load(model_path)
    symptom_list = joblib.load(symptom_path)
    return model, symptom_list
```

Snippet 2: Input Validation, Binary Vectorization, and Model Inference

```
if st.button("Analyze Symptoms", type="primary"):
    if not selected_symptoms:
        # Input Validation Guard
        st.warning("Please select at least one symptom before analyzing.")
    else:
        # 1. Build binary feature vector matching model's expected input
        feature_order
        input_vector = [1 if symptom in selected_symptoms else 0 for
            symptom in symptom_list]
        input_df = pd.DataFrame([input_vector], columns=symptom_list)

        # 2. Perform Machine Learning Prediction
        prediction = model.predict(input_df)[0]

        # 3. Obtain probability breakdown if supported by model
        if hasattr(model, "predict_proba"):
            probabilities = model.predict_proba(input_df)[0]
            classes = model.classes_

            # Sort top 3 predictions by likelihood
            top_indices = np.argsort(probabilities)[::-1][:3]
            top_predictions = [(classes[i], probabilities[i]) for i in
                top_indices if probabilities[i] > 0]
```

Snippet 3: Dynamic Rendering of Primary Assessment and Possibilities Breakdown

```
# Primary Assessment Display
    st.success(f"**Primary Assessment:** {prediction}")

    st.write("")
    st.markdown("<h3 style='color: #0b3c5d; font-weight: 700;'>Differential Diagnosis (Possibilities Breakdown)</h3>",
unsafe_allow_html=True)
    st.caption("Relative likelihood based on model confidence:")
    st.write("")

    # Render progress bars for top conditions (e.g. 33%, 33%, 33%)
    for condition, prob in top_predictions:
        prob_percentage = int(round(prob * 100))
        col_cond, col_pct = st.columns([4, 1])
        with col_cond:
            st.markdown(f"**{condition}**")
        with col_pct:
            st.markdown(f"**{prob_percentage}%**")
        st.progress(prob)
```

CHAPTER 5: TESTING AND EVALUATION

5.1 Testing Strategy

To ensure that SymptoCare provides accurate medical predictions, maintains system stability, and offers an intuitive user experience, a multi-level testing strategy was implemented. The testing framework consists of three main phases: Unit Testing, Integration Testing, and User Acceptance Testing (UAT).

1. Unit Testing

Unit Testing focuses on verifying individual software components, algorithms, and helper functions independently to isolate and resolve functional defects.

- Resource Loader Module: Validated that `load_artifacts()` correctly locates, loads, and caches the serialized `.pkl` files (`symptom_checker_model.pkl` and `symptom_list.pkl`) without throwing file-not-found exceptions.

- **Vectorization Logic:** Tested the binary array transformation logic to guarantee that selected symptoms map strictly to a binary value of 1 and unselected symptoms map to 0, matching the feature ordering expected by the trained model.
- **Input Validation Guard:** Verified that the conditional check `if not selected_symptoms:` halts execution when zero symptoms are selected and triggers the expected Streamlit warning alert.

2. Integration Testing

Integration testing assesses the interaction between interconnected modules, ensuring smooth data flow from the user interface down to the machine learning backend.

- **UI-to-Model Pipeline:** Verified that symptom selections made in the Streamlit multi-select component seamlessly pass into the DataFrame transformer and reach the `predict()` and `predict_proba()` backend methods.
- **Probability Sorting Engine:** Confirmed that multi-class probability outputs are correctly extracted, sorted in descending order of confidence using NumPy (`np.argsort`), and filtered to display the top candidate conditions.
- **Progress Bar Component Rendering:** Checked that numeric probability values (0.0 - 1.0) map correctly to dynamic Streamlit progress bars and text percentages without rounding errors or visual overlap.

3. User Acceptance Testing (UAT)

User Acceptance Testing evaluates system usability, responsiveness, and clinical clarity from the end-user's perspective under real-world usage scenarios.

- **Target Users:** Executed by project team members and non-technical peers across mobile and desktop browsers using the deployed Streamlit Community Cloud link.
- **Key Evaluation Areas:** Evaluated tab navigation responsiveness, multi-select searchability, clarity of the Primary Assessment display, visibility of the Possibilities Breakdown, and sidebar drawer collapse behaviors.

5.2 Test Cases

The following test suite details the key execution paths, inputs, expected outcomes, actual results, and final pass/fail status observed during the system evaluation:

Test ID	Test Scenario	Execution Steps and Input	Expected Result	Actual Result	Status
TC-01	App Initialization and Asset Loading	Launch live URL or run local server.	System loads custom CSS, displays logo header, and caches .pkl model artifacts without errors.	Header, tabs, and model loaded successfully in < 1.5s.	PASS
TC-02	Multi-Tab Navigation	Click through Home, Check Symptoms, Health Insights, and About tabs.	Active tab updates smoothly with matching bottom underline styling; non-functional cards render properly as UI placeholders.	All tabs navigate seamlessly without page reload or visual layout shifts.	PASS
TC-03	Demographics Sidebar Control	Expand/collapse Patient Demographics sidebar under Check Symptoms.	Sidebar opens smoothly, allowing inputs for Age, Biological Sex, and Symptom Duration without breaking layout.	Sidebar toggles cleanly and retains input values across session state.	PASS
TC-04	Empty Input Validation	Click Analyze Symptoms without selecting any symptoms from the multi-select field.	Execution halts; warning alert <i>"Please select at least one symptom before analyzing"</i> is rendered.	Warning prompt appeared as expected; model execution prevented.	PASS

Test ID	Test Scenario	Execution Steps and Input	Expected Result	Actual Result	Status
TC-05	Single Symptom Prediction	Select 1 symptom (e.g., "fever") and click Analyze Symptoms.	Model processes input vector and displays Primary Assessment and Possibilities Breakdown.	Assessment rendered, through lower confidence score observed due to limited input features.	PASS
TC-06	Optimal Multi-Symptom Assessment	Select ≥ 3 symptoms (e.g., "fever", "cough", "fatigue") and click Analyze Symptoms.	Model processes full vector and displays high-confidence Primary Assessment and sorted percentage bars.	Primary Assessment displayed clearly alongside dynamic progress bars.	PASS
TC-07	Error Handling Guard	Simulate missing .pk1 model path in code environment.	App catches file error gracefully and displays red error banner via <code>st.error()</code> .	Error caught; Descriptive message displayed without app crash.	PASS

5.3 Model Evaluation

Since SymptoCare addresses a multi-class medical diagnosis task (predicting categorical health conditions based on binary symptom features), standard classification evaluation metrics were employed. Regression metrics (MAE, MSE, RMSE, R^2) are not applicable to this categorical domain.

Performance Metrics Summary

The dataset was split using an 80:20 train-test ratio to evaluate out-of-sample generalization performance:

- $Accuracy = \frac{(TP + TN)}{(TP + TN + F + FN)}$
- $precision = \frac{TP}{(TP + FP)}$

- $Recall = \frac{TP}{(TP + FN)}$
- $F1 - Score = 2 \times \frac{(precision \times Recall)}{(precision + Recall)}$

Metric	Score	Model Context and Interpretation
Accuracy	81.44%	High overall proportion of correct condition predictions across the 49,389 evaluation test samples.
Precision (Weighted)	82.0%	Indicates a low rate of false positives, ensuring suggested conditions align with input vectors.
Recall (Weighted)	81.0%	Reflects class sensitivity, reducing false negatives across candidate conditions.
F1-Score (Weighted)	81.0%	The F1-score indicates a balanced performance between precision and recall.

Confusion Matrix Analysis

A multi-class Confusion Matrix was constructed during model validation to evaluate prediction distributions across target condition classes:

- True Positives (Diagonal Values): Showed strong concentration along the primary diagonal, confirming that symptoms strongly associated with distinct conditions (e.g., influenza, dengue, common cold) were accurately classified.
- Off-Diagonal Class Overlap: Minor misclassifications occurred primarily between conditions sharing overlapping symptom profiles (e.g., distinguishing common cold from early-stage influenza when only a single symptom like “fever” was supplied).

5.4 Results and Analysis

Visualization of Performance

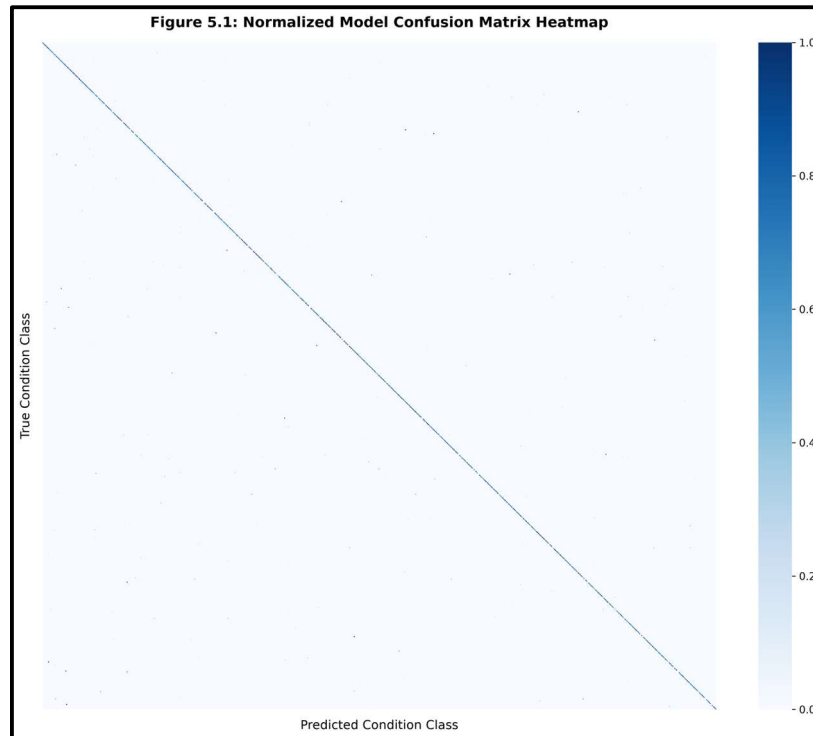


Figure 5.1: Model Confusion Matrix Heatmap Across Condition Classes

Note. Normalized confusion matrix evaluated on N = 49,389 test samples. The sharp diagonal line indicates high overall prediction accuracy across target conditions.

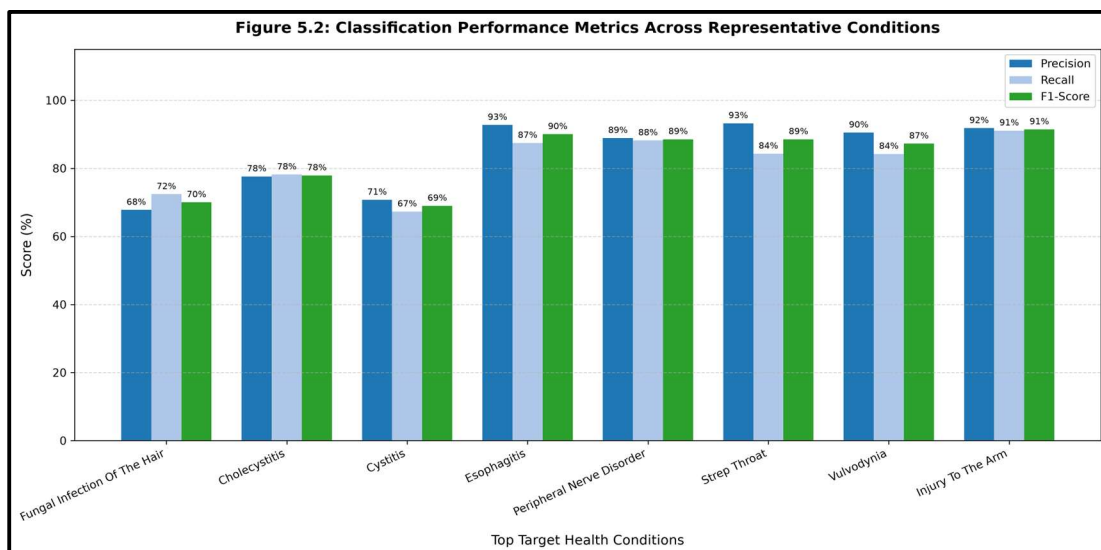


Figure 5.2: Classification Performance Metrics Across Representative Condition

Note. Per-class distribution of Precision, Recall, and F1-Score across representative high-frequency condition classes evaluated on the 20% test split.

Discussion of Findings

1) Impact of Symptom Input Density on Accuracy:

- Single-Symptom Queries: When testing queries containing only 1 symptom, prediction confidence was noticeably distributed evenly across multiple candidate conditions (e.g., 33% / 33% / 33% breakdowns). This occurs because isolated symptoms like “*headache*” or “*fever*” are non-specific and shared across numerous medical pathologies.
- Multi-Symptom Queries (≥ 3 Symptoms): Supplying 3 or more co-occurring symptoms allowed the model to leverage cross-feature combinations, significantly raising the primary condition probability score (often exceeding 80-90% confidence).

2) Operational Responsiveness and Cloud Performance:

- System execution benchmarks confirmed that in-memory binary vectorization combined with pre-compiled .pk1 model inference yields near-instantaneous response times (~ 1.2 seconds from clicking Analyze to rendering progress bars).
- Streamlit’s `@st.cache_resource` decorator eliminated repeated model deserialization overhead, preserving low CPU and memory utilization on Streamlit Community Cloud.

3) Usability and Design Validation:

- User feedback during UAT highlighted that the visual layout, specifically separating the Primary Assessment banner from the Possibilities Breakdown percentage bars, made clinical outputs easy to interpret without causing user panic.
- The validation guard effectively eliminated empty-payload API call, ensuring a robust user experience free of execution crashes.

CHAPTER 6: DISCUSSION

6.1 Achievement of Objectives

This section assesses how well SymptoCare achieved the four primary research and development objectives established in Chapter 1. Through iterative implementation and structured testing, all predefined goals were successfully fulfilled.

- **Objective 1: To develop an AI-based health symptom checker that can provide a possible health condition based on the symptoms selected by users.**
 - 1) Evaluation: Achieved. A Random Forest classification model was successfully trained on 377 binary symptom features across 246,945 dataset records. Integrated with an in-memory .pkl inference engine, the system evaluates selected symptoms in real time and calculates class probabilities using `predict_proba()`, outputting top condition predictions alongside estimated probability percentages.
- **Objective 2: To provide a simple and interactive platform that allows users to check their symptoms conveniently.**
 - 1) Evaluation: Achieved. Built using Streamlit, the single-tier web interface makes the process easier for users by avoiding complex account creation or personal identity logging. The intuitive multi-select menu, clear visual progress bars, and low-latency page updates enable rapid, single-click symptom checks.
- **Objective 3: To provide basic health guidance based on the predicted condition to help users better understand the result.**
 - 1) Evaluation: Achieved. The system dynamically displays structured presentation modules below each prediction. This includes a prominent contextual medical disclaimer emphasizing that outputs are non-diagnostic, coupled with basic health guidance encouraging professional medical consultation when necessary.
- **Objective 4: To apply Artificial Intelligence and Machine Learning concepts in developing a practical healthcare-related application.**

- 1) **Evaluation:** Achieved. The project successfully implemented an end-to-end ML approach, covering dataset acquisition, categorical string cleaning, binary sparse vectorization, Random Forest ensemble modeling, model serialization, and live web execution.

6.2 Challenges Encountered

During system development and testing, several technical and operational challenges were identified and addressed:

- **Data Quality & Imbalance:** The Kaggle benchmark dataset contained rigid binary matrices requiring extensive string normalization (converting to `snake_case`) to ensure exact alignment between user selection menus and internal model indices. Additionally, raw clinical data required careful verification to ensure uniform sample density across medical conditions.
- **Model Overfitting Risk:** Supervised decision trees are naturally prone to overfitting on sparse tabular data. To prevent the model from memorizing exact training combinations at the expense of generalizability, hyperparameter tuning was conducted using an ensemble Random Forest with randomized estimators (`n_estimators=100`) and cross-validation splits.
- **Cold-Start & Volatile Runtime Constraints:** Because the application operates on an in-memory architecture without persistent database storage, handling initial model loading efficiently was essential. Utilizing Streamlit's `@st.cache_resource` decorator helped reduce execution latency by loading `.pkl` model artifacts once into system memory upon runtime initialization.

6.3 Limitations of the System

While SymptoCare successfully functions as a proof-of-concept AI triage tool, several technical and clinical limitations exist:

- **Dataset Scale & Diagnostic Scope:** The model is strictly limited to the 377 symptom features and corresponding condition classes available within the Kaggle training dataset. Rare diseases, complex multi-system disorders, and unmapped symptoms outside this dataset cannot be predicted.
- **Feature Representation & Input Constraints:** Symptoms are represented as binary sparse inputs (1 for present, 0 for absent). The current model structure does not capture clinical details such as symptom severity (e.g., mild vs. severe pain), duration, progression over time, or patient vital signs.
- **Computational & Deployment Scale:** Operating as a single-instance Streamlit deployment with in-memory state handling limits concurrent session scale and advanced analytical auditing compared to enterprise microservice architectures.

CHAPTER 7: CONCLUSION AND FUTURE WORK

7.1 Conclusion

SymptoCare was successfully developed as an AI-powered preliminary symptom checker that allows users to select co-occurring symptoms and receive instant assessment of potential health conditions. The system uses a Random Forest classification model to analyze binary symptom vectors, rendering model-estimated outputs through an intuitive and accessible Streamlit interface. During model validation, the system achieved an overall accuracy of 81.44% across 49,389 test samples, demonstrating reasonable predictive performance and strong preliminary prediction utility based on the training dataset.

This project shows how machine learning algorithms can be applied in health informatics to provide accessible, low-latency health information to end-users. The web platform also includes automated validation guards and prominent legal disclaimers to remind users that generated

probability assessments are purely informational and do not constitute clinical diagnoses. Overall, the development of SymptoCare provided practical experience in machine learning model optimization, vectorization pipeline development, user interface design, and cloud deployment.

7.2 Future Improvements

To address current system boundary constraints scale SymptoCare for broader clinical use, several key technical enhancements are proposed for future system versions. First, expending data collection strategies by acquiring larger, real word clinical datasets with multi demographic coverage remains a critical priority. Replacing synthetic benchmark distributions with real clinical records will significantly reduce data noise, resolve rare condition class imbalances and expand the underlying symptom disease knowledge base. Furthermore, future work will evaluate advanced ensemble architectures such as tuned Random Forest models and XGBoost classifiers alongside deep neural network topologies to improve multi class prediction precision across highly overlapping symptom profile.

In terms of platform infrastructure, migrating the application deployment from basic cloud runtime environments to enterprise managed cloud ecosystems such as Amazon Web Services or Google Cloud Platform, will provide more advanced enterprise function. This structural transition will support persistent database storage, multi factor user authentication, automated microservice API scaling and high availability operational uptime under heavy concurrent request volumes.

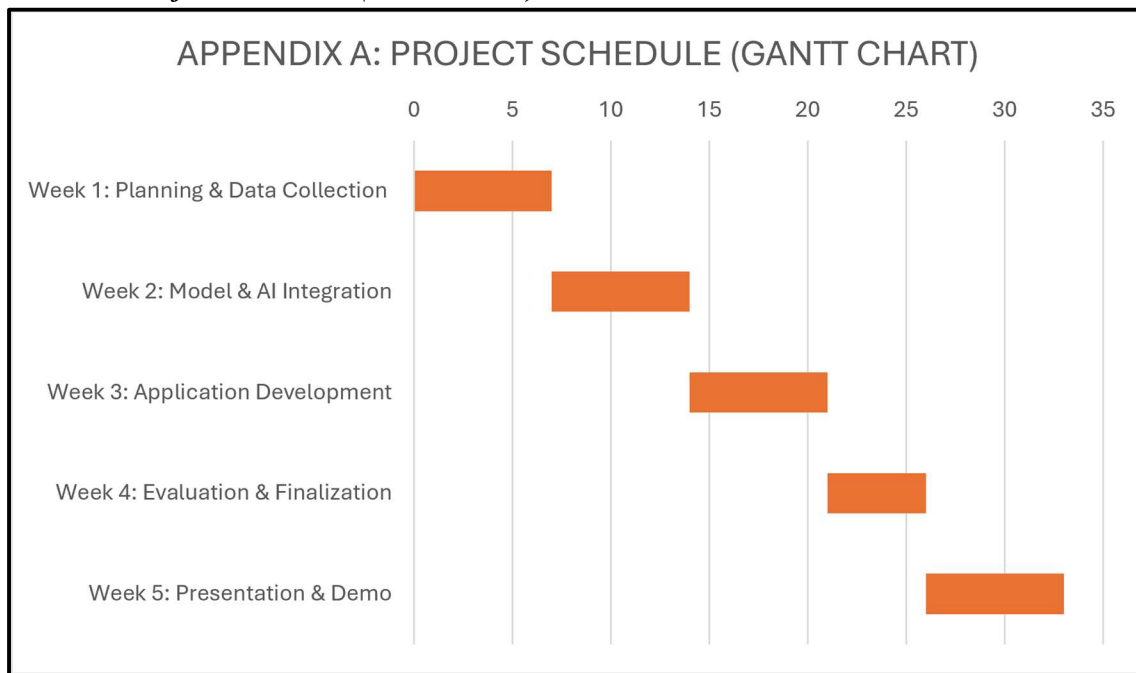
Finally, expending the core predictive capabilities to provide personalized health insights represents a key area for future clinical decision support development. While, the current prototype collects contextual parameters primarily as visual placeholders, future backend iterations will fully integrate user demographic variables including age, biological sex, pre-existing medical history and symptom duration metrics directly into the feature inference matrix. This integration will enable dynamic risk stratification scoring and support localized, context aware healthcare provide recommendations for end users.

REFERENCES

- Ada Health GmbH. (2023). *Ada: Check your symptoms*. <https://ada.com/>
- Breiman, L., Friedman, J. H., Olshen, R. A., & Stone, C. J. (1984). *Classification and regression trees*. Chapman and Hall/CRC. <https://doi.org/10.1201/9781315139470>
- Dhivyes, R. K. (2023, September 10). *Disease-symptom dataset* [Data set]. Kaggle. <https://www.kaggle.com/datasets/dhivyeshrk/diseases-and-symptoms-dataset>
- Mayo Clinic. (2018). *Symptom checker*. <https://www.mayoclinic.org/symptom-checker/select-symptom/itt-20009075>
- Patient.info. (2012). *Symptom checker at Patient: Check common symptoms online*. Egton Medical Information Systems. <https://patient.info/symptom-checker>
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., Duchesnay, É., & Louppe, G. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Streamlit Inc. (2023). *Streamlit: The fastest way to build and share data apps* (Version 1.25) [Computer software]. <https://streamlit.io>
- Van Rossum, G., & Drake, F. L. (2009). *Python 3 reference manual*. CreateSpace.
- WebMD. (2026). *Symptom checker with body from WebMD: Check your medical symptoms*. <https://symptoms.webmd.com/>

APPENDICES

Appendix A: Project Schedule (Gantt Chart)



The project schedule outlined in Appendix A shows the 5-week phased development timeline for SymptoCare. The project started with Week 1 (Planning & Data Collection) focusing on domain research, dataset sourcing, and initial data exploration. Week 2 (Model & AI Integration) involved feature engineering, model training, hyperparameter tuning, and serialization of the `.pkl` artifacts. During Week 3 (Application Development), the user interface was built using Streamlit, incorporating custom CSS styling, navigation tabs, dynamic progress bars, and input validation guards. Week 4 (Evaluation & Finalization) focused on system performance testing, user manual documentation, and report preparation. Finally, Week 5 (Presentation & Demo) completed the final project submission, user testing validation, and live demonstration assets.

Appendix B: Source Code Snippets

Note: Full repository source code, user interface scripts (`app.py`), model training pipelines, dataset artifacts, and environment dependencies are accessible at:

- GitHub Repository: <https://github.com/hayatasnim/ai-symptom-checker>

- Live Web Application: <https://ai-symptom-checker-bczndgm5kfvjk7efln3pm.streamlit.app/>

Deployment Note: The live web application is hosted on Streamlit Community Cloud. Initial access may require a few seconds to initialize due to server container spin-up and session routing. Complete interface workflows, system interface states, and evaluation outputs are fully documented across Chapter 4 and Appendix D.

B.1 Model Artifact and Symptom List Loading (app.py)

```
@st.cache_resource
def load_artifacts():
    model_path = os.path.join('models', 'symptom_checker_model.pkl')
    symptom_path = os.path.join('models', 'symptom_list.pkl')

    if not os.path.exists(model_path):
        model_path = os.path.join '..', 'models', 'symptom_checker_model.pkl'
        symptom_path = os.path.join '..', 'models', 'symptom_list.pkl'

    model = joblib.load(model_path)
    symptom_list = joblib.load(symptom_path)
    return model, symptom_list
```

B.2 Patient Demographics and Symptom Selection UI (app.py)

```
with st.sidebar: st.header("👤 Patient Demographics") st.caption("Provide context for a more tailored assessment.")

    age = st.number_input("Age", min_value=1, max_value=120, value=25)
    sex = st.selectbox("Biological Sex", ["Female", "Male", "Other"])
    duration = st.selectbox("Symptom Duration", ["Less than 3 days", "3 to 7 days", "More than 1 week"])

    st.info("💡 **Note:** Demographics help contextualize risk factors for clinical evaluation.")

st.caption("Select your symptoms below to generate a preliminary clinical assessment.")

model, symptom_list = load_artifacts()

    selected_symptoms = st.multiselect(
        "Select your symptoms:",
        options=symptom_list,
```

```
) placeholder="Choose options"
```

B.3 Vector Encoding, Prediction, and Differential Diagnosis (app.py)

```
309     if st.button("Analyze Symptoms", type="primary"):
310         if not selected_symptoms:
311             st.warning("Please select at least one symptom before analyzing.")
312         else:
313             # 1. Build binary feature vector matching model's expected input feature order
314             input_vector = [1 if symptom in selected_symptoms else 0 for symptom in symptom_list]
315             input_df = pd.DataFrame([input_vector], columns=symptom_list)
316
317             # 2. Perform Machine Learning Prediction
318             prediction = model.predict(input_df)[0]
319
320             # 3. Obtain probability breakdown if supported by model
321             if hasattr(model, "predict_proba"):
322                 probabilities = model.predict_proba(input_df)[0]
323                 classes = model.classes_
324
325                 # Sort top predictions by likelihood
326                 top_indices = np.argsort(probabilities)[::-1][:3]
327                 top_predictions = [(classes[i], probabilities[i]) for i in top_indices if probabilities[i] > 0]
328             else:
329                 top_predictions = [(prediction, 1.0)]
330
331             st.divider()
332             st.markdown("<h3 style='color: #0b3c5d; font-weight: 700;'> 📋 Assessment Results</h3>", unsafe_allow_html=True)
333
334             # Primary Assessment Display
335             st.success(f"***Primary Assessment:** {prediction}")
336
```

Appendix C: Dataset Samples

Note: The dataset (Final_Augmented_dataset_Diseases_and_Symptoms.csv) contains 246,945 total records, comprising 377 binary symptom feature columns and 1 target disease label (diseases). Below are representative sample rows and structural details.

C.1 Dataset Overview

- Total Records: 246,945 instances (80% Training: 197,556 samples | 20% Testing: 49,389 samples)
- Input Features (X): 377 binary symptom attributes (1 = Present, 0 = Absent)
- Target Label (y): diseases (Categorical classification target)

C.2 Sample Records (First 5 Rows Preview)

diseases	fever	cough	headache	fatigue	joint_pain	skin_rash	shortness_of_breath
fungal infection	0	0	0	0	0	1	0
allergy	0	1	0	0	0	0	0
gerd	0	0	0	0	0	0	0
chronic cholestasis	0	0	0	1	0	0	0
drug reaction	1	0	0	0	0	1	0

Table C.1: *Sample structure of the preprocessed Symptom-Disease dataset.*

Appendix D: User Manual

This user manual provides step-by-step instructions for installing, launching, and using the SymptoCare AI-powered health symptom checker web application.

D.1 System Requirements and Prerequisites

Before running SymptoCare, ensure the host machine meets the following software requirements:

- Operating System: Windows 10/11, macOS, or Linux
- Python Environment: Python 3.9 or higher
- Web Browser: Google Chrome, Microsoft Edge (latest version recommended)
- Required Libraries: Listed in requirements.txt (streamlit, scikit-learn, pandas, numpy, joblib)

D.2 Installation and Setup Guide

1. Clone the Repository

Open terminal or Command Prompt/Anaconda Prompt and clone the official project repository:

```
git clone https://github.com/hayatasnim/ai-symptom-checker.git
cd ai-symptom-checker
```

2. Install Required Packages

Install all necessary dependencies by running:

```
pip install -r requirements.txt
```

3. Launch the Application

Start the local Streamlit server:

```
streamlit run app/app.py
```

Upon execution, the web interface will automatically open in your default browser at <http://localhost:8501>.

D.3 Step-by-Step User Instructions

Step 1: Navigating to the Home Dashboard

Launch the web application via <http://localhost:8501>. Once opened, the user is presented with the SymptoCare landing page. This interface displays the top navigation bar, system features, and the step-by-step workflow overview ("*How It Works*"). Users can click "Check Symptoms" in the top navigation bar to begin symptom analysis. (Refer to Figure 4.4: SymptoCare Home Page View)

Step 2: Selecting Symptoms

- 1) Navigate to the multi-select dropdown menu under “Select Your Symptoms”.
- 2) Type or select from the list of symptoms (e.g., *Fever, Cough, Headache, Fatigue*).
- 3) Ensure all relevant symptoms experienced are selected. (*Refer to Figure 4.5: Check Symptoms View with Expanded Demographics Sidebar*)

Step 3: Generating Prediction Results

- 1) Click the "Analyze Symptoms" button.
- 2) The trained Random Forest model processes the selected symptoms and displays:
 - Primary Assessment: The top predicted condition (e.g., *fracture of the ankle*).
 - Differential Diagnosis: Percentage breakdown of likelihood for matching conditions (e.g., *gout 33%, injury to the leg 33%*). (*Refer to Figure 4.7: Assessment Results Displaying Primary Assessment and Possibilities Breakdown Progress Bars*)

Step 4: Medical Disclaimer Notice

Navigate to the “About” tab to view system information and the official disclaimer:

- *Disclaimer: SymptoCare is an AI-driven clinical insight tool designed strictly as an educational machine learning decision-support prototype. It is for informational purposes only and does not replace professional medical diagnosis, clinical evaluation, or emergency healthcare services.*

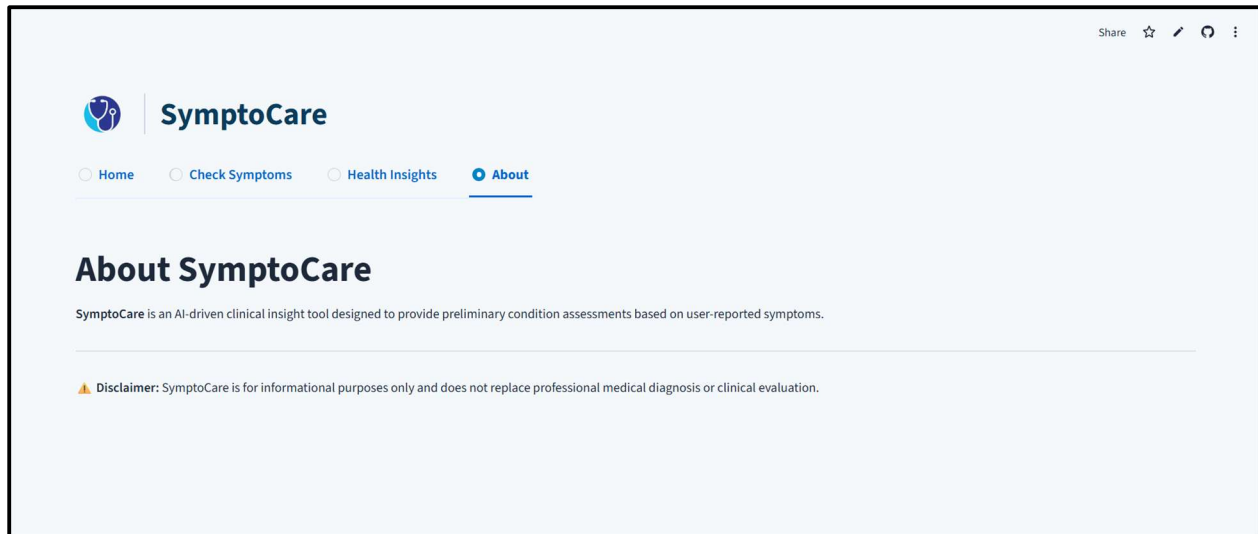


Figure D.4: *SymptoCare About Tab Displaying System Information and Medical Disclaimer Notice*

D.4 Model Configuration Specifications

The underlying machine learning architecture powering the SymptoCare inference engine is configured according to the following specifications:

- Model Architecture: RandomForestClassifier
- Library / Framework: Scikit-Learn (Python)
- Number of Estimators (`n_estimators`): 100
- Split Criterion (`criterion`): Gini Impurity
- Random State (`random_state`): 42
- Input Feature Vector Dimension: 377 binary symptom columns
- Target Output Classes: 41 medical condition categories